SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Containerization and Cloud-Native Applications

GUIDED BY
Dr.R. LATHA

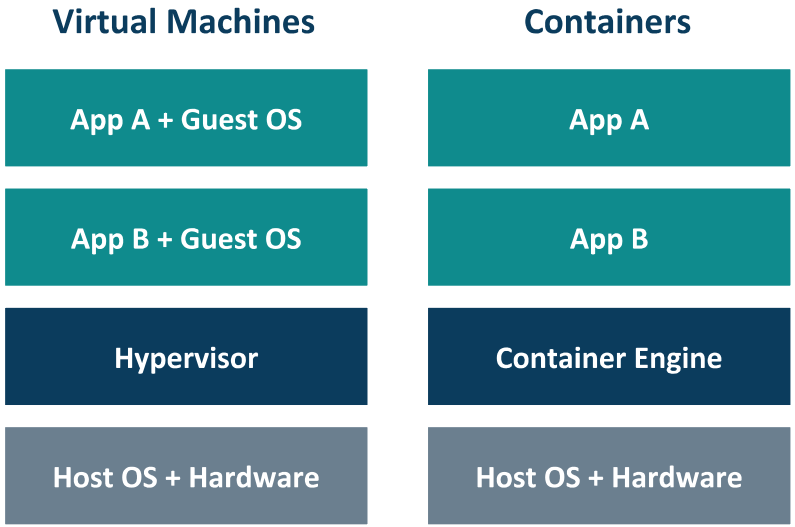
PRESENTED BY
B.V.THARUN

Containers • Architecture • Benefits • Real-World Deployment

What Is a Container?

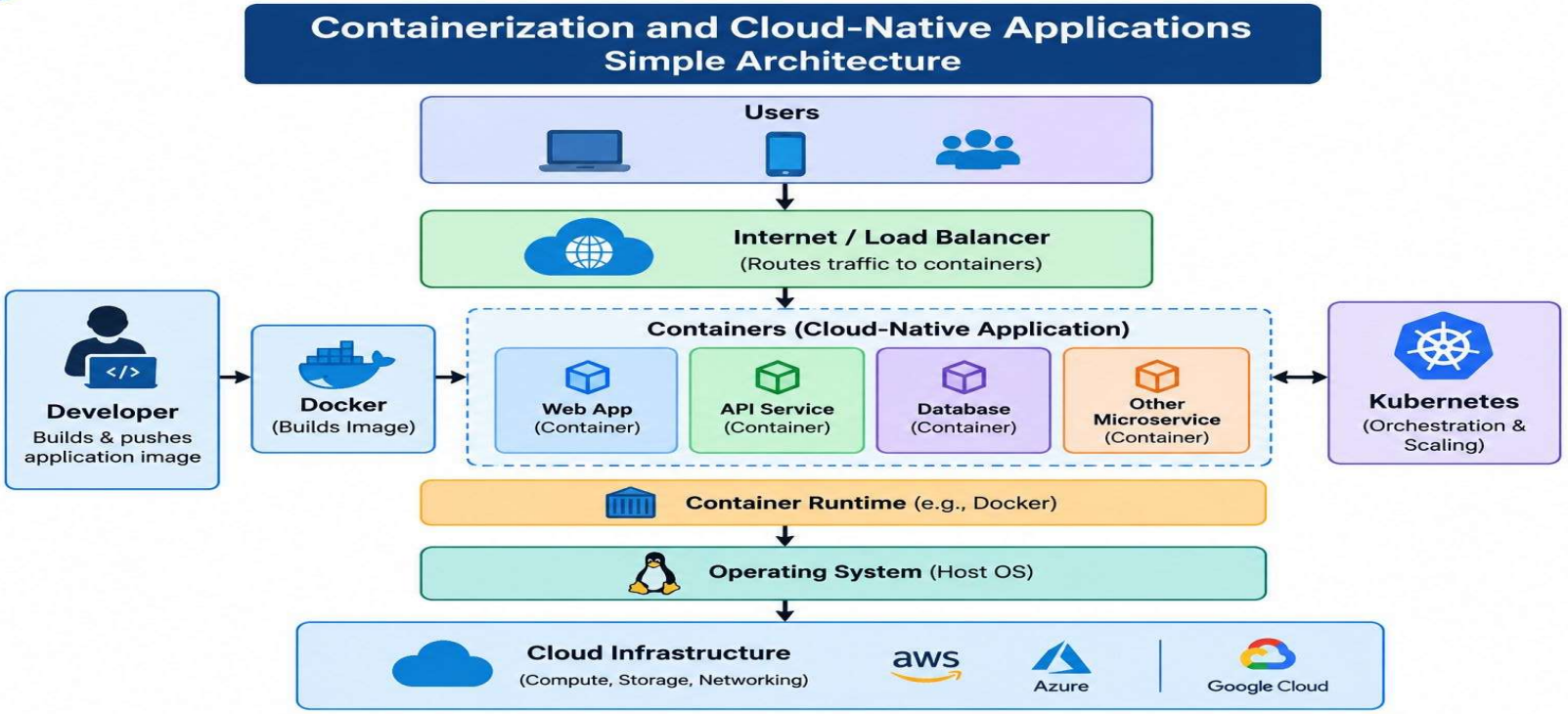
A **container** is a lightweight, isolated environment that packages an **application, its code, libraries, and dependencies** together so that it can run consistently on any system.

- Shares the host OS kernel
- Isolated using namespaces and cgroups
- Starts in seconds, runs the same everywhere



No guest OS per app → smaller, faster, denser

Container Architecture



Key Benefits of Containers

Portability

Build once, run on laptop, data center or any cloud.

Efficiency

Share the kernel: less overhead, higher server density.

Speed

Start in seconds; faster builds, tests and releases.

Scalability

Scale replicas up or down automatically on demand.

Consistency

Same image in dev, test and production ends "works on my machine".

Isolation

Faults and dependencies stay contained per service.

Cloud-Native Applications

Applications designed from the start to exploit the cloud: elastic, resilient and quick to change.

1

Microservices

Small services, each owned and deployed independently

2

Containers

Packaging unit that runs identically anywhere

3

Orchestration

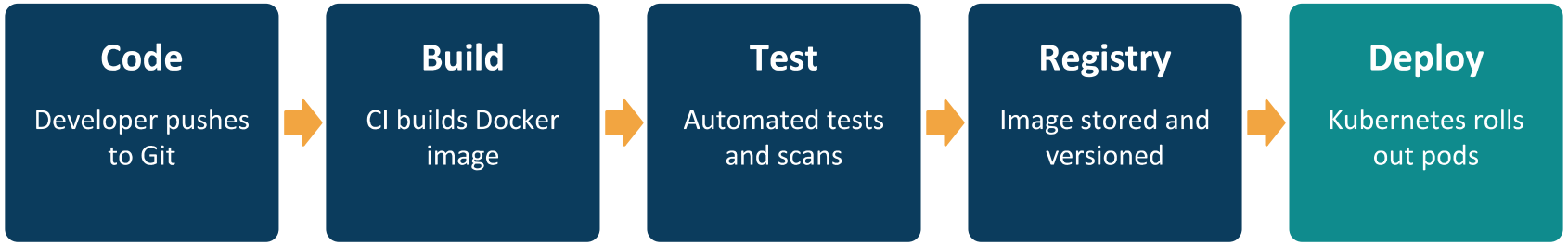
Kubernetes schedules, scales and self-heals

4

DevOps + CI/CD

Automated pipelines ship changes safely and often

Real-World Deployment Example



Services running as containers

- Frontend, Catalog, Cart, Payment, Orders
- Each service has its own image and release cycle
- Load balancer routes traffic to healthy pods

Outcome

- Autoscaling handles sale-day traffic spikes
- Rolling updates give zero-downtime releases
- Failed pods restart automatically

Challenges and Best Practices

Challenges

Security of images and registries

Networking and storage complexity

Orchestration learning curve

Monitoring many short-lived services

Best Practices

Use minimal, trusted base images

Scan images and run as non-root

Define resource limits and health checks

Automate with CI/CD and observability



Key Takeaways



- Containers package apps for consistent, portable deployment
- Cloud-native design pairs microservices, containers and orchestration
- CI/CD plus Kubernetes delivers fast, resilient, scalable releases

Thank You